Completing 4D Snake with Masked PPO and a Hamiltonian Reverse Curriculum

Repository: https://github.com/BurnyCoder/4d-snake-rl - all numbers below come from the run artifacts copied to `reports/data/` and the per-experiment write-ups in `reports/experiments/`.

Abstract

We build an N-dimensional snake game (default 4D, 4^4 = 256 cells, 8 moves) whose rules are implemented once as vectorised numpy over a body-age grid, so thousands of boards step together in a single process (126k-235k environment steps per second, 33k PPO steps per second end to end on a laptop GPU). We train sb3-contrib MaskablePPO agents to *complete* the game - fill every cell - with a legal-action mask from the environment, a sparse reward (+1 food, -1 death, +10 win, -0.001 per step) and a Backplay-style reverse curriculum whose demonstration is a Hamiltonian route (a cycle for even sizes; for odd sizes a cycle over all cells but one corner with a guarded detour). Scripted route following completes 2^4, 3^4 and 4^4 in 100 % of episodes and defines the step-count ceiling (66 / 1,875 / 16,448 steps). On 2^4 plain PPO plateaus at 0.88-0.93 completion; a strictly gated curriculum (advance only at 90 % success) reaches 0.963 at the same 5M-step budget while completing in 43.5 steps, a third fewer than the route follower's 65.9. On the larger boards the result is negative within our budgets: on 3^4 (odd parity) every arm plateaus at 54-57 % fill with no completions, and on 4^4 a 100M-step curriculum run moves the start frontier from 255 to 199 cells and then collapses, reaching 40 % fill from the true start (best episode 66 %) and zero completions. We attribute the collapse to schedules tied to the total budget rather than to curriculum progress, and to the lack of transfer from cycle-shaped starts to true starts. Behaviour-cloning the same network on 262k route-follower decisions spread over all fill levels (about ten seconds of supervised training) yields a neural policy that completes the full 4^4 board in 100 % of deterministic evaluation episodes (3 seeds x 100), at the follower's 16,448 steps; PPO fine-tuning of this clone is the route to a faster learned completer.

1. Introduction

Snake is a coverage problem in disguise: to win, the body must at the end form a Hamiltonian path of the grid, and every intermediate state must keep such a path reachable. In four dimensions the grid graph has degree 8, the intuition of "corners" disappears, and a projected 4D board stops being playable by a human beyond a certain snake length (eugeneko/Snake4D's own README). This project asks whether a model-free policy can learn to complete the 4D board, and builds the full pipeline needed to answer it: game, human play, scripted baselines, batched training, evaluation protocol and reports.

2. Related work

- **Masked PPO for snake.** linyiLYi/snake-ai fills a 12x12 board with MaskablePPO, a body gradient in the observation, and decaying learning rate and clip range; we adopt the masking, the body-age channel and the schedules, but not its raw distance bonus.
- **Invalid-action masking.** Huang and Ontanon (2020) justify masking theoretically and show its value grows with the number of invalid actions - eight moves in 4D make it

essential.

- **Reverse curricula.** Backplay (Resnick et al. 2018) starts near the end of a demonstration and moves the start backwards; Salimans and Chen (2018) gate the move on the success rate. Our demonstration is a Hamiltonian route rather than a human trajectory.
- **Hamiltonian-cycle agents.** twanvl/snake benchmarks cycle followers and perturbed cycles (0 % losses); the perturbed-cycle rule that the head must never overtake the tail is from johnflux's Nokia-snake write-up (2015). Every rectangular grid graph with an even number of cells has a Hamiltonian cycle (Itai, Papadimitriou and Szwarcfiter 1982); `hamilton.ham_cycle` lifts a 2D cycle to d dimensions and verifies the result, and the odd-board cycle over all cells but a corner is our own construction, also verified.
- **Imitation.** Behaviour cloning (Pomerleau 1988) learns the expert's action from the observation; its covariate-shift failure mode and the DAgger remedy are due to Ross, Gordon and Bagnell (2011).
- **AlphaSnake** (Du et al. 2022) is, by its authors' account, the first published learned agent with a >50 % win rate on a 10x10 board, using MCTS; it frames snake as a Hamiltonian-cycle problem exactly as we do.
- **Shaping.** Ng, Harada and Russell (1999): potential-based shaping is the only shaping that cannot change the optimal policy; we keep it available but off.

3. The game

State: a body-age grid `age[cell]` (0 empty, 1 tail, `length` head), head and food indices. A step resolves the target cell, decrements every age unless the snake eats (so the tail vacates first and following the tail is legal), tests occupancy, writes the head, checks `length == C` before spawning food uniformly over free cells. Starvation (more than `4C` consecutive idle steps) and an absolute cap (`C^2` steps) truncate the episode; the truncating step is paid `r_step` like any other and there is no death penalty. Observation: `4C + 2` floats (body, time-to-vacate `age/length`, head one-hot, food one-hot, `length/C`, hunger). Mask: inside the board, free after the tail moves, not the neck; an all-false row falls back to in-bounds moves because MaskablePPO masks with `−1e8`, not `−inf`. Rendering folds the 4D board into a z-by-w grid of x-by-y tiles.

Parity: the grid graph is bipartite, so a Hamiltonian cycle needs an even cell count. 4^4 has one (constructed recursively and verified); 3^4 (41/40 colour split) has none, and completing it requires the final body to be a Hamiltonian path with both ends in the larger colour class.

4. Method

Agent. MaskablePPO, MLP [512, 512] for policy and value, gamma 0.99, GAE 0.95, learning rate 3e-4 -> 1e-5 and clip range 0.2 -> 0.05 (both linear), entropy 0.01, target KL 0.03, 4 epochs, minibatch 8192, 4096 batched environments with 64-step rollouts (262,144 samples per update). Defaults were chosen by the throughput benchmark (exp00) and the reference agent's schedules.

Batched environment. One `SnakeBatch` holds all boards; a custom SB3 `VecEnv` exposes it and `VecMonitor` records episodes. The benchmark measured 10x higher PPO throughput than `DummyVecEnv/SubprocVecEnv` over single environments and showed the PPO update, not the environment, to be the bottleneck (minibatch 2048 -> 8192 lifts throughput 19k -> 31k fps on the best row; `reports/data/exp00_benchmark/benchmark.json`).

Curriculum. 80 % of resets place the snake as a segment of length `U[hi − 4, hi]` along a random rotation of the Hamiltonian route, starting at `hi = C − 1`; the frontier moves back by `max(1, C/64)` cells whenever at least 500 curriculum episodes reached 90 % success; 20 % of resets always start from length 1 and evaluation always starts from length 1.

Baselines. The route follower (Hamiltonian cycle, or cycle-minus-corner with a detour that skips at most as many cycle cells as are free) and uniformly random legal play.

Evaluation. 100 episodes x 3 seeds, deterministic and stochastic, masked `evaluate_policy`; metrics: completion rate, mean fill, steps to complete, win-within-K.

5. Experiments

Pre-registered arms live in `experiments/*.env`; every write-up follows question -> hypothesis -> setup -> results -> learnings:

- exp00 - throughput benchmark: batched env on CUDA, 4096 envs; minibatch 8192 gives 31k fps (33k at 16384; 8192 adopted for more gradient steps per rollout).
- exp01 - scripted baselines: route follower 100 % on 2^4 / 3^4 / 4^4 (65.9 / 1,874.5 / 16,447.8 steps); random legal play 29 % / 0 % / 0 % (`reports/data/exp01_*/summary.json`). The first odd-board follower (an open Gray-code path) reached about 4 % fill (run not archived, see exp01) and was replaced by the cycle-minus-corner construction.
- exp02 - 2^4: plain PPO 0.877 (5M) and 0.933 (20M) deterministic completion; loosely gated Backplay (rho 0.2) 0.850; strictly gated Backplay (rho 0.9, window 4) 0.963 at 5M with 43.5 steps per completion (`reports/data/exp02*_best/summary.json`). Failures are collisions with one cell left. The strict gate became the default.
- exp03 - 3^4 (81 cells, odd): plain PPO, strict Backplay (gate 0.9) and relaxed Backplay (gate 0.8) at 30M steps each all end at 0.54-0.57 fill and 0 completions; the curriculum frontier never left the endgame (80 -> 76-78) because the corner detour was never mastered above the gate (`reports/experiments/exp03_ppo_3x4.md`).
- exp04 - 4^4 (256 cells): Backplay (gate 0.8, window 16, step 8) for 100M steps / 54 min: seven frontier advances (255 -> 199) between 18.6M and 53.7M steps, none in the remaining 46M, and the curriculum success rate fell to 0 from about 75M steps as the learning rate, clip range and entropy decayed towards their end values; true-start fill 0.40 (best episode 0.66), completion 0.000 (`reports/experiments/exp04_ppo_4x4.md`, `reports/data/exp04_ppo_4x4*/`).
- exp05 - 4^4 imitation warm start: behaviour cloning of the route follower reaches 1.000 expert-action accuracy and the cloned network completes the board in 1.000 +- 0.000 of deterministic episodes (0.787 +- 0.041 when sampling), 16,448 steps per game. exp05b fine-tuned it with PPO for 20M steps (lr 1e-4, clip 0.1, no entropy, no curriculum): completion stayed at 1.00 in all nine evaluations but the step count did not move and `approx_kl` stayed below 2e-5 throughout - a clone whose expert-move probability is about 0.9998 (from the final cloning loss) gives PPO nothing to compare, so fine-tuning needs deliberate exploration (`reports/experiments/exp05_bc_4x4.md`, `reports/data/exp05*`).

The generated cross-run table is `reports/all_experiments.md`.

6. Discussion

On the 16-cell board, masking plus a strictly gated reverse curriculum is what turns "eats a

lot" into "fills the board": the loose gate advanced the frontier before endgames were mastered and was indistinguishable from no curriculum, while the strict gate gave the best policy at the same budget. The remaining failures there are endgame traps (boxed in with one free cell), a global-planning constraint that a one-hot MLP learns only partially from terminal rewards.

On 81 and 256 cells the same recipe fails, for two identifiable reasons. First, the curriculum's starting states - snakes laid along the Hamiltonian cycle - are not states the true-start policy ever reaches (the distribution shift inherent to reverse curricula, Florensa et al. 2017), so skill on curriculum starts (0.88-0.9 fill, 60-85 % endgame success) did not transfer to the true start (0.4-0.55 fill). Second, the learning-rate, clip-range and entropy schedules decay with the total budget, not with curriculum progress, so the 4^4 agent lost the ability to adapt exactly when the frontier reached the hardest states and its endgame success fell to zero. The odd 3^4 board adds the parity constraint and a two-step corner detour that was never learned.

The imitation warm start (exp05) closes the feasibility gap: the route follower supplies unlimited (observation, action) pairs on every board, the states cover every fill level at once, and the resulting network completes the 256-cell board in every deterministic episode. What remains for RL is efficiency - the clone plays the 16,448-step cycle, while the learned 2^4 agent showed that PPO can cut a third of the steps by taking shortcuts - and robustness under sampling. Other candidates for the from-scratch setting are schedules driven by the frontier rather than the step count, longer budgets with smaller frontier steps, and decision-time search with the learned value function (AlphaZero-style policy iteration, Silver et al. 2017), as in AlphaSnake.

7. Conclusion

The repository delivers a playable 4D snake, a batched training stack that runs 33k PPO steps per second on one laptop GPU, scripted proofs that every board size is completable (with a new cycle-minus-corner construction for odd sizes), a reproducible experiment loop, and two results: model-free MaskablePPO with a Hamiltonian reverse curriculum completes the 16-cell 4D board in 96 % of episodes but not 3^4 or 4^4 within 30M and 100M steps; a neural network behaviour-cloned from the Hamiltonian follower completes the full 4^4 board in 100 % of deterministic episodes, and PPO fine-tuning from that clone is the path to a faster learned completer.

References

- Huang, S. and Ontanon, S. (2020). A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. arXiv:2006.14171.
- Resnick, C. et al. (2018). Backplay: "Man muss immer umkehren". arXiv:1807.06919.
- Salimans, T. and Chen, R. (2018). Learning Montezuma's Revenge from a Single Demonstration. arXiv:1812.03381.
- Ng, A., Harada, D. and Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. ICML, 278-287. https://dl.acm.org/doi/10.5555/645528.657613
- Grzes, M. (2017). Reward Shaping in Episodic Reinforcement Learning. AAMAS. https://dl.acm.org/doi/10.5555/3091125.3091208
- Itai, A., Papadimitriou, C. H. and Szwarcfiter, J. L. (1982). Hamilton Paths in Grid Graphs. SIAM J. Comput. 11(4), 676-686. https://doi.org/10.1137/0211056
- Du, Y. et al. (2022). AlphaSnake: Policy Iteration on a Nondeterministic NP-hard

Markov Decision Process. arXiv:2211.09622.
- Andrychowicz, M. et al. (2020). What Matters in On-Policy Reinforcement Learning? arXiv:2006.05990.
- Raffin, A. et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR 22(268), 1-8. https://jmlr.org/papers/v22/20-1364.html
- Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Towers, M. et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032.
- Pomerleau, D. A. (1988). ALVINN: An Autonomous Land Vehicle in a Neural Network. NeurIPS. https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html
- Ross, S., Gordon, G. and Bagnell, D. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (DAgger). AISTATS. arXiv:1011.0686.
- Muller, R., Kornblith, S. and Hinton, G. (2019). When Does Label Smoothing Help? NeurIPS. arXiv:1906.02629.
- Florensa, C. et al. (2017). Reverse Curriculum Generation for Reinforcement Learning. CoRL. arXiv:1707.05300.
- Silver, D. et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815.
- Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html
- johnflux (2015). Nokia 6110 Part 3 - Algorithms. https://johnflux.com/2015/05/02/nokia-6110-part-3-algorithms/
- linyiLYi/snake-ai, twanvl/snake, instadeepai/jumanji, Pella86/Snake4d (see THIRD_PARTY_NOTICES.md).

All experiments

Generated by `uv run snake4d report` from the artifacts in `runs/` (small copies in `reports/data/`). Each experiment's question, hypothesis, analysis and learnings are in its own file:

Metrics by run

run	phase	board	timesteps	wall_min	fps	fill_final
exp01_route_2x4	evaluate	2^4				
exp01_random_2x4	evaluate	2^4				
exp01_random_3x4	evaluate	3^4				
exp01_route_4x4	evaluate	4^4				
exp02_ppo_2x4	train	2^4	5,013,504	2.167	38,526	0.98
exp01_route_3x4	evaluate	3^4				
exp02b_ppo_2x4_backplay	train	2^4	5,013,504	2.3	36,208	0.983
exp01_random_4x4	evaluate	4^4				
exp02c_ppo_2x4_long	train	2^4	20,021,248	8.567	38,901	0.986
exp02d_ppo_2x4_backplay_strict	train	2^4	5,013,504	2.117	39,204	0.994
exp03a_ppo_3x4_nocur	train	3^4	30,015,488	9.9	50,518	0.548
exp02_ppo_2x4_best	evaluate	2^4				
exp02b_ppo_2x4_backplay_best	evaluate	2^4				
exp02c_ppo_2x4_long_best	evaluate	2^4				
exp02d_ppo_2x4_backplay_strict_best	evaluate	2^4				
exp03b_ppo_3x4_backplay	train	3^4	30,015,488	9.7	51,486	0.909
exp03a_ppo_3x4_nocur_best	evaluate	3^4				
exp04_ppo_4x4	train	4^4	100,139,008	53.667	31,096	0.708

exp03c_ppo_3x4_backplay_relaxed	train	3^4	30,015,488	10.633	47,033	0.898
exp03b_ppo_3x4_backplay_best	evaluate	3^4				
exp03c_ppo_3x4_backplay_relaxed_best	evaluate	3^4				
exp04_ppo_4x4_best	evaluate	4^4				
exp05_bc_4x4	imitate	4^4				
exp05_bc_4x4_eval	evaluate	4^4				
exp05b_ppo_4x4_from_bc	train	4^4	20,185,088	22.517	14,930	0.191
exp05b_ppo_4x4_from_bc_best	evaluate	4^4				
exp00_benchmark	bench	4^4				

Figures: `reports/figures/<run>_curves.png` and `<run>_fill_hist.png` per training run.

Experiment 00 - Throughput benchmark (which environment/device configuration to train with)

Date: 2026-09-05 (re-measured with the minibatch sweep built into `snake4d bench`; the first grid of 2026-09-04 gave the same ordering). Command: `uv run snake4d bench --set batch_size=2048 --set run_name=exp00_benchmark` (run `20260905-010410_bench_exp00_benchmark`). Every number below is read from reports/data/exp00_benchmark/benchmark.json; the table docs/benchmark.md is generated from the same file.

Question

On this laptop (16 logical CPUs, torch using 8 threads, NVIDIA GeForce RTX 5070 Laptop GPU with 8.0 GB, torch 2.14.0+cu130; versions.json), which of the candidate vectorisation schemes gives the most MaskablePPO training steps per second on the 4^4 board, and what budgets follow?

Hypothesis

The research brief predicted, but could not cite a measurement for, the ordering batched single-process env > > DummyVecEnv > > SubprocVecEnv for a cheap environment like snake: per-env Python overhead dominates single envs and inter-process communication dominates subprocesses (SB3 docs on `SubprocVecEnv`, https://stable-baselines3.readthedocs.io/en/master/guide/vec_envs.html; Gymnasium paper Fig. 1, https://arxiv.org/abs/2407.17032; openai/baselines#608, https://github.com/openai/baselines/issues/608).

Setup

- Board 4^4 (256 cells, observation 1026 floats), MLP [512, 512], `n_epochs=4`, `batch_size=2048` for the grid.
- Grid rows: batched `SnakeVecEnv` with 256 / 1024 / 4096 envs (`n_steps=64`); `DummyVecEnv(16)` and `SubprocVecEnv(16)` over single `SnakeEnv`s (`n_steps=512`); each on cpu and cuda, with `torch.set_num_threads` 1 and 8; about 200k-262k timesteps per row.
- Minibatch sweep: the best grid row re-run with minibatch 2048 / 8192 / 16384.
- fps = `model.num_timesteps / wall time of learn()` (SB3's own `time/fps` is cumulative).
- Env-only rows step the batched env with random legal actions and no policy.

Results

configuration	fps	notes
batched 4096 envs, cuda, batch 2048	20,433	best row of the grid

batched 1024 envs, cuda, batch 2048	18,843	
batched 256 envs, cuda, batch 2048	14,577	
batched 4096 envs, cpu, 8 threads	5,954	1 thread: 1,612
DummyVecEnv(16), best of cpu/ cuda	2,006	
SubprocVecEnv(16), best of cpu/ cuda	1,881	
env only, batched 256 / 1024 / 4096	234,823 / 136,074 / 126,304 steps/s	no policy

Minibatch sweep on the best row: batch 2048 -> 8192 -> 16384 gives 18,735 -> 30,686 -> 32,967 fps (best measured throughput 32,967 fps). Run-to-run variance is about 8 %: the batch-2048 setting measured 20,433 fps in the grid and 18,735 in the sweep.

Analysis and learnings

- The hypothesis holds with a wide margin: the batched env trains 10x faster than the better of the two SB3 vectorisers, and `SubprocVecEnv` is no better than `DummyVecEnv` here (spawned workers on Windows spend their time on pickling and pipes, not on the microsecond-scale env step).
- The environment is not the bottleneck: it steps at 126k-235k steps/s (4.3-7.9 microseconds per env-step) but PPO reaches 20k at batch 2048. The minibatch sweep shows the update dominates: 4 epochs x 128 minibatches = 512 gradient steps per 262,144-sample rollout at batch 2048, each with fixed Python overhead; batch 8192 (128 gradient steps) gives 31k fps and 16384 (64 gradient steps) 33k fps with diminishing returns, so 8192 was adopted.
- Env-only throughput falls as the batch grows (235k at 256 envs to 126k at 4096) because the observation tensor (n_envs x 1026 float32, 16 MB at 4096) is rebuilt every step; the cost is memory bandwidth, not the game rules.
- CPU with one torch thread is slow (2k fps) because the 1026 -> 512 -> 512 forward pass and the update run single-threaded; 8 threads help 3.7x, the GPU another 3.4x. Training on CPU is a fallback only.
- Larger env batches beat smaller ones on cuda (15k -> 20k from 256 to 4096 envs): the per-step fixed costs (get_action_masks list, VecMonitor loop, buffer add) are amortised over more envs.

Decisions

- Train on the batched env with `SNAKE_N_ENVS=4096`, `SNAKE_BATCH_SIZE=8192`, `SNAKE_DEVICE=auto` (cuda), `SNAKE_TORCH_THREADS=8`; these are the defaults in `.env.example` and `config.py`.
- Budget arithmetic at ~31k fps: 10M steps = 5 min, 100M steps = 54 min (the real 100M-step run of exp04 took 54 min at 31.1k fps with its evaluations included).
- Evaluation cadence: `SNAKE_EVAL_EVERY=2,097,152` (8 rollouts of 262,144, about a minute) by default; the generated recommendation in docs/benchmark.md targets one evaluation per ten minutes instead and is deliberately not adopted, because the small boards finish long before that.

Experiment 01 - Scripted baselines: is every board completable, and at what step cost?

Date: 2026-09-04. Command: `uv run snake4d evaluate --set size=<2|3|4> --set ndim=4 --set policy=<route|random>` (runs `*_evaluate_exp01_*`; per-episode CSVs and summaries in `reports/data/exp01_*`).

Question

Can a scripted policy fill the 2^4, 3^4 and 4^4 boards from a random length-1 start under the episode caps used for training (starvation after more than `4 * cells` consecutive idle steps, `cells^2` total), and how many steps does it need? What does masked random play achieve? These two numbers bracket every learned agent: the route follower is the completability proof and the efficiency ceiling to beat; random legal play is the floor.

Hypotheses

- H1: following a Hamiltonian cycle completes every even board (4^4 = 256 cells, 2^4 = 16) in 100 % of episodes, in about `cells^2 / 2` steps (the head must circle the board once per food on average).
- H2: the odd board 3^4 (81 cells, colour classes 41/40, no Hamiltonian cycle) is also completable by a scripted policy, but a plain Hamiltonian *path* follower is not enough.
- H3: masked random play completes the 16-cell board sometimes and never the larger ones.

Setup

100 episodes x 3 seeds per policy and board, deterministic policies, the evaluation protocol of docs/evaluation.md (masked `evaluate_policy`, one batched env per episode, explicit seeds).

Results

board	policy	success (mean +- std over seeds)	fill	mean steps to complete	win within `C^2/2` steps
2^4 (16)	route	1.000 +- 0.000	1.000	65.9	100 %
2^4 (16)	random	0.293 +- 0.048	0.816	129.4	15 %
3^4 (81)	route	1.000 +- 0.000	1.000	1,874.5	100 %
3^4 (81)	random	0.000 +- 0.000	0.226	-	0 %
4^4 (256)	route	1.000 +- 0.000	1.000	16,447.8	100 %
4^4 (256)	random	0.000 +- 0.000	0.077	-	0 %

Analysis and learnings

- H1 confirmed: the cycle follower never fails on even boards. The step cost is close to `C^2 / 4` (16,448 for C = 256), half the `C^2 / 2` worst case, because the food is on average half a lap ahead of the head. This is the efficiency bar: a learned agent should complete 4^4 in far fewer than ~16k steps.
- H2 confirmed only after a fix. The first version of the route follower used the reflected Gray-code *path* on odd boards and reached only about 4 % fill on 3^4 (run not archived; that follower was removed in PR #6, commit f3b68d7) - worse than random play at 22 %: an open path has a dead end, so the head reaches the end, has to leave the route, and then keeps colliding with its own body near the end. The fix (`hamilton.cycle_minus_corner`) builds a Hamiltonian cycle over the 80 cells other than the corner - an explicit 2D construction plus a "ladder" splice of the corner's column into a neighbouring column for each extra dimension - and the policy enters the corner only when the food is there and at most as many cycle cells would be skipped as are currently free (otherwise the head could overtake its tail). With that, 3^4 is completed in 100 % of episodes with zero fallbacks, in 1,874.5 steps on average. The stale Gray-path run was removed from `runs/`; this paragraph is its record.
- H3 confirmed: masked random play finishes the 16-cell board in 29 % of episodes (masking alone prevents most deaths on a tiny board) but never the 81- or 256-cell boards, where it fills 23 % and 8 % before dying or starving.
- The starvation cap (`4 * cells`) is never binding for the route follower: the longest wait for food is one lap (`C` steps) plus the detour on odd boards.

Decisions

- The route follower is the demonstration for the Backplay curriculum on every board size, including odd ones (now a cycle segment, not a path prefix).
- Report every learned agent's steps-to-complete against 65.9 / 1,874.5 / 16,447.8 (`steps_to_complete_mean` in `reports/data/exp01_route_*/summary.json`).

Experiment 02 - MaskablePPO on the 2^4 board (16 cells): plain PPO vs curriculum variants

Date: 2026-09-04. Arms (each an `experiments/*.env` file, trained with `uv run snake4d train`, best checkpoint re-evaluated with `uv run snake4d evaluate --set model_path=...`):

arm	curriculum	budget	wall
exp02 `exp02_ppo_2x4`	off	5M steps	2.2 min
exp02b `exp02b_ppo_2x4_backplay`	Backplay, `rho=0.2`, window 8, min 200 episodes	5M	2.3 min
exp02c `exp02c_ppo_2x4_long`	off	20M	8.6 min
exp02d `exp02d_ppo_2x4_backplay_strict`	Backplay, `rho=0.9`, window 4, min 500 episodes	5M	2.1 min

Common settings: 1024 batched envs, `n_steps=32`, `batch_size=4096`, 4 epochs, MLP [512, 512], gamma 0.99, lr 3e-4 -> 1e-5, clip 0.2 -> 0.05, `ent_coef` 0.01, evaluation of 100 true-start episodes every 327,680 steps (655,360 for exp02c), CUDA (36-39k steps/s, final `time/fps`).

Question

Does MaskablePPO alone learn to *complete* the smallest 4D board, how fast, and does the Backplay reverse curriculum change the outcome?

Hypotheses

- H1: plain PPO reaches 100 % completion within a few hundred thousand steps (the board is tiny).
- H2: the curriculum is unnecessary on 16 cells (exp02b ~ exp02).

Results (best checkpoint, 100 episodes x 3 seeds)

arm	deterministic success	stochastic success	steps to complete	best in-training eval (at step)
exp02 plain 5M	0.877 +- 0.041	0.853 +- 0.034	34.4	0.91 (2.9M)
exp02b Backplay rho 0.2	0.850 +- 0.029	0.843 +- 0.026	34.6	0.91 (3.6M)
exp02c plain 20M	0.933 +- 0.005	0.863 +- 0.025	35.7	0.96 (17.0M)

exp02d Backplay rho 0.9	**0.963 +- 0.009**	**0.960 +- 0.033**	43.5	**1.00 (3.6M)**
route follower (exp01)	1.000	-	65.9	-
random legal (exp01)	0.293	-	129.4	-

Figures: `reports/figures/exp02*_curves.png` (fill, eval success, frontier, episode length) and `exp02*_fill_hist.png`.

Analysis and learnings

- H1 is **rejected**. Plain PPO climbs quickly (eval success 0.41 after 0.3M steps, 0.8 after 1.3M) but plateaus around 0.85-0.93; 4x more steps only lifts the best checkpoint from 0.88 to 0.93. Rollout fill is 0.98, so the agent almost always reaches 15/16 cells.
- Failure mode: of 61 failed evaluation episodes of the 20M model, 35 end at fill 15/16 by a collision (median length 33 for those 35, 31 over all 61 failures; far below the 64-step starvation cap; `eval_episodes.csv`), i.e. the snake boxes itself in with one cell left. Completing the last cell requires the body to be a Hamiltonian path whose head can still reach the free cell - a global constraint the one-hot MLP policy only partly learns from sparse terminal rewards.
- H2 is rejected in an instructive way. With the loose gate (`rho=0.2`) the frontier ran from 15 to 1 in about 15 seconds of training - the curriculum advanced as soon as one fifth of the endgame episodes succeeded, so it taught nothing (exp02b = exp02 within noise). With a strict gate (`rho=0.9`, narrow window, 500 episodes per decision) the agent had to master each endgame frontier before moving back; the frontier still reached 1 within a minute, but the resulting policy is the best of all arms (0.963 deterministic, 0.960 stochastic, an in-training evaluation of 1.00 at 3.6M steps) at the same 5M budget. The strict gate is now the default (`SNAKE_CURRICULUM_RHO=0.9`, `WINDOW=4`, `MIN_EPS=500`).
- Deterministic (argmax) evaluation beats sampling for every arm except exp02d, where both agree: the strict-curriculum policy is confident in the endgame; the others rely on argmax to avoid their own exploration noise.
- Steps to complete (34.4-43.5) are a third to a half below the route follower's 65.9: the learned agents cut across the board instead of circling it, which is the efficiency gain RL buys.
- The eval curve is noisy at 100 episodes (about +-3 percentage points, the binomial standard error at p = 0.9 and n = 100 - an estimate, not a measurement), which is why the reported numbers use the best checkpoint re-evaluated with 300 episodes; the in-training "1.00" of exp02d was one such 100-episode sample.

Decisions

- Curriculum defaults switched to the strict gate for exp03b (3^4) and exp04 (4^4).
- Open question carried forward: closing the last few percent on tiny boards likely needs either longer training with the strict gate, a lower final entropy, or an endgame-aware feature (reachability of the free region); tested next on the harder boards first.

Experiment 03 - MaskablePPO on the 3^4 board (81 cells, odd parity)

Date: 2026-09-04. Arms (30M steps each, 2048 batched envs, `n_steps=64`, batch 8192, ~10 min each on CUDA):

arm	curriculum	frontier reached	wall
exp03a `exp03a_ppo_3x4_nocur`	off	-	9.9 min
exp03b `exp03b_ppo_3x4_backplay`	Backplay, strict gate `rho=0.9`, window 4, step 1	80 -> 78	9.7 min
exp03c `exp03c_ppo_3x4_backplay_relaxed`	Backplay, `rho=0.8`, window 8, step 4	80 -> 76	10.6 min

The route on this board is the 80-cell cycle minus the corner (docs/game_rules.md), so the frontier starts at 79 effective (the callback logged 80 until the read-back fix in the review PR).

Question

Does PPO complete an odd board, where the last cell must be reached through the corner detour and the final body must be a Hamiltonian path with both ends in the majority colour class? Does the reverse curriculum that worked on 2^4 carry over?

Hypotheses

- H1: without a curriculum, fill plateaus well below 1 with no completions (parity dead-ends).
- H2: the strict-gate Backplay curriculum reaches the true start and yields completions.

Results (best checkpoint, 100 episodes x 3 seeds, deterministic)

arm	success	fill (eval)	true-start fill during training (mean / best episode)	curriculum-start success
exp03a plain	0.000	0.566	0.540 / 0.914 (74/81)	-
exp03b strict gate	0.000	0.548	0.529 / 0.877 (71/81)	0.78 mean; cleared the 0.9 gate twice (0.90 -> 79 at 14.9M, 0.91 -> 78 at 15.1M), peak 0.907
exp03c relaxed gate	0.000	0.542	0.528 / 0.877 (71/81)	0.60

route follower (exp01)	1.000	1.000	-	-

Figures: `reports/figures/exp03*_curves.png`, `exp03*_fill_hist.png`.

Analysis and learnings

- H1 confirmed: every arm fills 54-57 % of the board on average and at most 74/81 cells (exp03a, 161,535 true-start episodes) or 71/81 (exp03b/c, ~80k each; `episodes.json`); no completion. Episodes end by collision at fill ~0.5-0.6, long before the starvation cap.
- H2 rejected: the curriculum never left the endgame. Starting as a 76-80-cell cycle segment, the agent succeeded in 60-85 % of episodes on average and cleared the 90 % gate only twice (frontier 80 -> 78, then a plateau around 0.83); relaxing the gate to 80 % with larger steps moved the frontier only to 76 and lowered the curriculum success to 60 %. Completing from a near-full odd board needs the corner detour (enter the corner only when the food is there and the skipped arc is free) - a two-step plan the policy did not learn reliably in 30M steps.
- The plain and curriculum arms end with the same true-start fill (0.53-0.55): the curriculum episodes (80 % of the batch) did not transfer to the true start at all, so the 20 % true-start episodes drove the same learning as in exp03a with a fifth of the data.
- The odd board is therefore not just "harder": it changes the endgame from "follow the free arc" to "follow the free arc and time a detour", and PPO with this observation does not get there.

Decisions

- Treat 3^4 as the parity stress test, not a stepping stone: 4^4 (even, exp04) is the target.
- Next steps if the odd board is pursued: warm-start the policy by imitating the route follower (behaviour cloning on states from route rollouts), then fine-tune with PPO; or add an explicit observation feature for the corner detour condition.

Experiment 04 - MaskablePPO with Backplay on the 4^4 board (256 cells): the headline run

Date: 2026-09-04. `experiments/exp04_ppo_4x4.env`: 4096 batched envs, `n_steps=64`, batch 8192, 100M steps in 54 minutes on CUDA (31.1k steps/s, the final cumulative `time/fps`); Backplay along the 256-cell Hamiltonian cycle with `rho=0.8`, window 16, frontier step 8, 500 episodes per gate decision (relaxed from the 2^4 defaults after exp03b stalled at a 90 % gate).

Question

Can model-free MaskablePPO learn to fill the 4D board from length 1 within a 100M-step budget when episodes are seeded along the Hamiltonian cycle and the start moves backwards as the agent masters the endgame?

Hypothesis

The frontier moves from 255 to 1 within the budget (32 advances of 8 cells at the 80 % gate, each needing at least 500 curriculum episodes), and the true-start completion rate becomes positive once the frontier passes roughly half the board.

Results

metric	value
curriculum frontier	255 -> 247 (18.6M steps, 9.1 min) -> 239 (22.3M) -> 231 (22.5M) -> 223 (40.6M) -> 215 (46.4M) -> 207 (46.9M) -> 199 (53.7M, 27.1 min); no advance in the remaining 46.4M steps (`run.log`, `progress.csv`)
curriculum-start success at frontier 199	0.81 at the advance (53.7M), 0.60 at 56.1M, **0.00 from ~75M to 100M steps**
rollout fill (all starts)	0.81 at 26M, falling to 0.71 at 100M
true-start fill (training)	mean 0.377, best episode 0.660 (169/256 cells); 81,995 true-start episodes of 624 steps on average (`episodes.json`)
true-start completion (eval, 47 evaluations of 100 episodes)	0.000 throughout
best checkpoint, 100 episodes x 3 seeds, deterministic / stochastic	success 0.000 / 0.000, fill 0.401 / 0.395
route follower (exp01)	success 1.000, 16,448 steps

Figures: `reports/figures/exp04_ppo_4x4_curves.png`, `exp04_ppo_4x4_fill_hist.png`.

Analysis and learnings

- **Negative result.** The agent did not complete the 4^4 board; the best true-start episode filled two thirds of it. The board is completable (the route follower proves it), so this is a learning failure, not a feasibility one.
- **The curriculum collapsed instead of advancing.** Seven advances happened between 18.6M and 53.7M steps (gate success 0.80-0.84 each time), then the frontier stayed at 199 and the success rate of the 57-cell-free endgame *fell to zero* in the last quarter of training. The learning rate decays linearly to 1e-5 over the whole 100M budget regardless of curriculum progress, so by the time the frontier reached the hardest states the policy could no longer adapt; the entropy bonus and the clip range decay the same way. Tying the schedules to the total budget was a design error for a curriculum whose pace is unknown in advance.
- **Distribution shift is visible.** Fill on curriculum starts (0.88, `episodes.json`) never transferred to the true start (0.38-0.40): a snake laid along the cycle is in a state the true-start policy never produces, and the true-start episodes (20 % of resets) learned no faster than plain PPO did on the smaller boards.
- **What the agent did learn**: from length 1 it eats about 100 cells (eval fill 0.40) before colliding - 5.2x the random floor (0.077) - in episodes of roughly 570-620 steps (evaluation 566, training 624), i.e. it learned greedy, mostly safe foraging, not the long-horizon coverage strategy.

Decisions and next steps

1. Decouple the schedules from the curriculum: constant learning rate, clip range and entropy until the frontier reaches 1, then decay; or make the decay a function of the frontier.
2. Warm-start by imitation: behaviour-clone the policy on (observation, action) pairs from the route follower (which is available in unlimited quantity from the batched env), then fine-tune with MaskablePPO - the network then starts from a policy that already completes the board and RL only has to make it faster. This is the most promising route to a learned completer.
3. Smaller frontier steps (4) with a longer budget (300M+ steps, ~3 h), and evaluation of the curriculum-start success on a fixed set of frontier states instead of the moving window.
4. As a fallback, search at decision time (AlphaSnake-style MCTS with the learned value function).

Experiment 05 - Imitation warm start on 4^4: behaviour cloning, then PPO fine-tuning

Date: 2026-09-04. Arms: `experiments/exp05_bc_4x4.env` (`snake4d imitate`) and `experiments/exp05b_ppo_4x4_from_bc.env` (`snake4d train --set model_path=<exp05>/bc_model.zip`).

Question

exp04 showed that model-free PPO with a reverse curriculum does not complete the 256-cell board in 100M steps, while the scripted route follower completes it every time. Can the *same* policy network (MLP [512, 512] on the `4C + 2` observation) learn to complete the board by imitating the route follower, and does PPO fine-tuning then keep the completion rate while reducing the ~16k steps per game?

Hypotheses

- H1: the network can represent the route follower on 4^4 (the action is a function of the head cell only, a 256-entry lookup) and behaviour cloning reaches near-100 % expert-action accuracy.
- H2: the cloned policy completes the board from the true start in 100 % of deterministic evaluations; sampled actions occasionally leave the cycle and lower the rate.
- H3: PPO fine-tuning with gentle settings (lr 1e-4 -> 1e-5, clip 0.1 -> 0.05, no entropy bonus, no curriculum) keeps completion near 100 % and shortens games by taking shortcuts.

Setup

- Data: 4096 batched envs x 64 steps = 262,144 (observation, expert action, mask) samples, starts spread uniformly over every snake length via curriculum-style resets along the cycle (`p_true_start = 0.2`), expert = `RoutePolicy` (Hamiltonian cycle follower).
- Cloning: negative log-likelihood of the expert action under the masked policy distribution, Adam lr 1e-3, minibatch 8192, 20 epochs; the value head is untouched. Wall time 9 s for the 20 epochs (12 s including data collection; `run.log`).
- Evaluation: docs/evaluation.md protocol, 100 episodes x 3 seeds, deterministic and stochastic.

Results

policy	deterministic success	stochastic success	fill (det / stoch)	steps to complete (det)
behaviour-cloned network (exp05)	**1.000 +- 0.000**	0.787 +- 0.041	1.000 / 0.901	16,448

clone + PPO fine-tuning, best checkpoint (exp05b)	**1.000 +- 0.000**	0.763 +- 0.017	1.000 / 0.884	16,414
route follower (exp01, scripted)	1.000	-	1.000	16,448
PPO + Backplay from scratch (exp04, 100M steps)	0.000	0.000	0.401 / 0.394	-

Expert-action accuracy after cloning: 1.000 (loss 2e-4 after 20 epochs).

PPO fine-tuning (exp05b): 20M steps, 22.5 min, from `bc_model.zip`

metric	value
in-training evaluations (100 true-start episodes every 2.1M steps)	success 1.00 at every one of the 9 evaluations
mean episode length at those evaluations	16,218 - 16,514 steps (unchanged; `evaluations.npz`)
`train/approx_kl`	7.9e-7 to 1.7e-5 at every update (prints as 0.000; `progress.csv`)
throughput	15k fps (half of exp04: every rollout ends no episode, and each evaluation is 100 x 16k steps)

H3 is **rejected in an instructive way**: the completion rate stayed at 100 %, but the policy did not change at all. Behaviour cloning drove the expert action's probability to about 0.9998 (exp(-2e-4) from the final cloning loss; not measured directly), so every sampled action equals the argmax, the advantage estimates carry no information about alternative moves, and with `ent_coef = 0` there is no force widening the distribution: PPO's update is negligible (approx_kl below 2e-5). Fine-tuning a near-deterministic clone needs exploration on purpose - an entropy bonus or a temperature on the cloned logits, or a less over-fitted clone (early stopping, label smoothing: Muller, Kornblith and Hinton 2019, https://arxiv.org/abs/1906.02629; or DAgger-style data aggregation: Ross, Gordon and Bagnell 2011, https://arxiv.org/abs/1011.0686) - which is the next experiment.

Analysis and learnings

- H1 and H2 confirmed: a neural network policy now completes the 4D board from the true start in every deterministic evaluation episode - the first learned agent in this project to do so on 256 cells. It is the route follower distilled into weights: identical step counts (16,448 on average, seed by seed), so it has learned the cycle as a head-position -> action lookup rather than any shortcut behaviour.
- Sampling actions (stochastic evaluation) completes 74-84 % of episodes: with 8 actions and a distribution that puts most but not all mass on the cycle move, a few thousand samples per game are enough for an off-cycle move that eventually traps the snake. Deterministic play is the right mode for this policy; fine-tuning should sharpen the distribution.
- Why imitation succeeds where the curriculum failed: the cloned policy is trained on states from *all* fill levels at once (uniform-length starts) with a dense supervised signal, so there is no frontier to advance and no distribution shift between "curriculum starts" and "true starts" - the true-start trajectory of the cycle follower is itself made of those states.
- Cost: about ten seconds of cloning versus 54 minutes of PPO in exp04; the expert is free

because the route follower runs inside the batched env.

Decisions

- The imitation warm start is the recommended path for larger boards: it is the only learned 4^4 completer so far.
- PPO's remaining job is efficiency (fewer steps than the cycle) and robustness under sampling. exp05b shows a zero-entropy clone cannot be fine-tuned as is; the next arm fine-tunes with `ent_coef > 0` (or clones with label smoothing) so that PPO has alternatives to evaluate, with the completion rate as a hard constraint (keep `best_model.zip` by evaluation success).